

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

Prima di cominciare

Prima di cominciare

La pratica del Modulo 2 si fa nel vostro Claude Code: quattro controlli, prima di M2-E01

```
# nel terminale Linux o WSL 2, nella cartella esercitazioni
git pull                        # gli elementi del Modulo 2
claude --version                # 2.1.293 o successiva
tools/verifica_ambiente.sh      # deve finire con: Tutto pronto
claude doctor                   # se qualcosa non torna
```

- `claude --version`: **2.1.293 o successiva** — nell'esercitazione il giudice gira su **Haiku**, e con le versioni precedenti l'alias `haiku` punta a un modello più vecchio
- In Claude Code, `/status`: il metodo di accesso attivo dev'essere il vostro **abbonamento Pro**
- **Nessuna** `ANTHROPIC_API_KEY`: se c'è, Claude Code la usa al posto dell'abbonamento — e con `claude -p`, sempre; l'uso dell'API non è incluso nel piano Pro
- `tools/verifica_ambiente.sh` **tutto verde**: controlla anche la chiave, senza mai stamparla, e Python, che serve a `misura.sh`; `claude doctor` è una diagnostica in sola lettura

Il ciclo di ogni esercizio

Preparare, lavorare in Claude Code, verificare — e, se serve, ricominciare

```
# dalla cartella esercitazioni
tools/prepara_esercizio.sh M2-E01   # lo spazio di lavoro
cd ~/corso-claude-code/M2-E01       # fuori dal repository
claude --model sonnet               # i passi del README
# poi /exit, e si torna nella cartella esercitazioni:
cd -
tools/verifica.sh M2-E01            # 0 = fatto, 1 = non ancora
tools/reset_esercizio.sh M2-E01     # da capo, se serve
```

1. **Prepara**: crea `~/corso-claude-code/M2-E01/`, fuori dal repository, con il punto di partenza; è un repository git con un primo commit, e `git status` e `git diff` mostrano che cosa avete cambiato
2. **Lavora**: `claude --model sonnet` nello spazio di lavoro, con i «Passi in Claude Code» del `README.md` dell'elemento
3. **Verifica**: esce con 0 se il risultato è quello atteso, con 1 se non ancora (e dice che cosa manca), con 2 se qualcosa impedisce la verifica
4. **Ricomincia**: cancella lo spazio di lavoro, dopo aver chiesto conferma, e lo prepara di nuovo

Gli strumenti della misura

Nello spazio di lavoro di M2-E01: un file per versione, una misura in tre passi, un giudice

- **Un file per versione**: `prompt/v0.md` è il prompt pessimo, `prompt/v1.md` sarà il vostro; `misura.sh` mette al posto di `{{messaggio}}` il file `dati/messaggio.md`, uguale per tutte le versioni — cambia solo il testo, e `git diff --no-index prompt/v0.md prompt/v1.md` mostra che cosa
- `! ./misura.sh v1`, **dalla sessione**: con `!` davanti il comando gira **senza passare da Claude**, e con il suo output entra nella conversazione — la **risposta** di Sonnet, con `claude -p`; i **controlli** scritti in codice (il telefono, i segnaposto, la lunghezza); il **giudizio**; su v0, un **secondo giudizio** della stessa risposta
- **Il giudice**: un subagent del progetto, `.claude/agents/giudice.md`, su **Haiku** — non il modello che si valuta; `misura.sh` lo chiama con `claude -p --agent giudice --json-schema`, e il giudice, per ogni affermazione verificabile di `rubrica.md`, scrive il motivo, poi se è rispettata. Ogni criterio porta la **pratica** che legge, e `registro.md` quella di ogni versione

Un ambiente verificato e uno strumento di misura sono metà dell'esercizio.

Il piano Pro, con misura

L’impronta d’uso di M2-E01, dal suo README: chiamate, modelli, effort

- Chiamate brevi, senza strumenti: **tre su v0** (la risposta e due giudizi), **due su v1**, cinque in tutto, sette con la rimisura — la risposta su **Sonnet**, i giudizi su **Haiku**, con effort **low**, più rapido ed economico
- Più la sessione su Sonnet di quattro o cinque turni; **niente Opus**, che pesa sensibilmente di più sul limite d’uso, e **niente Fable**, che sul piano Pro va solo con i crediti d’uso
- Il giudice gira in una sua chiamata `claude -p`, lanciata da `misura.sh`, non dalla sessione: attinge agli **stessi limiti**, e l’`usage` del suo JSON ne conta i token — un subagent lanciato **dentro** una sessione, invece, nell’`usage` di quella sessione non compare
- `/usage`: le barre dei limiti del piano, **condivisi** fra claude.ai e Claude Code; i limiti non sono un numero fisso di messaggi — dipendono da lunghezza, modello e funzioni